

Spring boot config server

Key features

- provides centralized configuration management for microservices in a distributed system.
- **Centralized management:** A single server manages configuration properties for all applications and environments (e.g., development, testing, production).
- **Version control:** By default, the config server uses a Git repository as its backend, which offers versioning, labeling, and auditing of configuration changes. Other backends like HashiCorp Vault, JDBC, and the local file system are also supported.
-

- **Dynamic updates:** It supports refreshing configurations at runtime without needing to restart the microservices. This is enabled by the Spring Boot Actuator `/actuator/refresh` endpoint and the `@RefreshScope` annotation.
- **Encrypted properties:** The server can symmetrically or asymmetrically encrypt and decrypt property values to secure sensitive data.
- **API-based access:** Configuration data is exposed through an HTTP API, making it accessible to any application that can make an HTTP request, regardless of language.

Config server project

- Create the config server project
 - Use the Spring Initializr tool (<https://start.spring.io/>) to create a new Spring Boot project.
 - Dependencies: Add Config Server.
- Generate: Download and extract the project.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-config-server</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Enable the config server

```
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.cloud.config.server.EnableConfigServer;
```

```
@EnableConfigServer
```

```
@SpringBootApplication
```

```
public class ConfigserverApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(ConfigserverApplication.class, args);  
    }  
}
```

Configure the Git repository

- In the `src/main/resources/application.properties` (or `application.yml`) file of the config server, specify the Git repository where the configuration files are stored.
- The server typically runs on port 8888 by convention.
- `application.properties`

application.properties

spring.application.name=product

spring.datasource.url=jdbc:mysql://localhost:3306/db3

spring.datasource.username=root

spring.datasource.password=root

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

#spring.jpa.hibernate.ddl-auto=create

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true

spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect

server.port=8080

Config client

In project add config client dependency

```
<dependency>  
    <groupId>org.springframework.cloud</groupId>  
    <artifactId>spring-cloud-starter-config</artifactId>  
</dependency>
```


application.properties

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

spring.config.import=optional:configserver:http://localhost:8082

management.endpoints.web.exposure.include=refresh

#spring.profiles.active=dev

spring.profiles.active=default